

Internship Daily Documentation

Day 5 — Friday, May 1, 2026

Smart Internship & Hiring Portal | Intern: Mageswaran G

Field	Details
Date	Friday, May 1, 2026
Intern	Mageswaran G
Project	Smart Internship & Hiring Portal
Mentor	Sandiya
Module	Week 1 — Module 1 Final Day (Day 5)
Day Goal	Fix browser refresh logout issue. Complete full auth lifecycle. Verify MongoDB data.
Final Status	Module 1 — 100% Complete

1. SOD (Start of Day) — Morning Plan

SOD mail sent to Sandiya. Today was focused on completing the one remaining gap in Module 1 — the auth lifecycle fix.

Planned Tasks:

- Fix refresh token flow — auto-login on page reload
- Debug and resolve backend /auth/refresh API returning 500 error
- Ensure HTTP-only cookie handled correctly across requests
- Validate complete auth lifecycle: Login → Refresh → Logout
- End-to-end testing of full authentication flow
- Begin Module 2 planning after completion

2. Problems Found and Fixed Today

Problem 1 — Browser Refresh Logs User Out

When user pressed Cmd+R on dashboard, the page went to login. Token was lost because React memory clears on refresh.

Root Cause:

- accessToken stored in React state (memory)
- Memory clears when browser refreshes
- No code to restore token after page reload

Fix Applied:

- Added useEffect in AuthContext.jsx to call /auth/refresh on app startup
- Backend reads HTTP-only cookie and returns new access token
- Token restored in memory automatically — user stays logged in

Problem 2 — /auth/refresh Returning 500 Error

After adding useEffect, the refresh endpoint crashed with 500 Internal Server Error.

Root Cause (found using DEBUG logs):

- User found by ID in MongoDB
- BUT: compareToken() called BEFORE checking if user exists
- If user is null → user.refreshToken crashes with: Cannot read properties of undefined

Fix Applied — authService.js:

- Moved null check BEFORE compareToken call
- Step 1: Find user
- Step 2: Check if user exists (if not → throw error)
- Step 3: Check if user.refreshToken exists
- Step 4: THEN call compareToken (safe now)

Problem 3 — Cookie Not Reaching Backend

Even after fix, refresh kept failing. Cookie was being blocked by browser.

Root Cause:

- Frontend was calling 127.0.0.1:8000 but browser was at localhost:5173
- Cookie was set on 127.0.0.1 domain
- sameSite: Strict blocked cookie from being sent cross-hostname

Fix Applied:

- Changed baseUrl in authService.js from 127.0.0.1:8000 to localhost:8000
- Changed sameSite: Strict to sameSite: Lax in authController.js cookie config
- Both frontend and backend now use same hostname — cookie flows correctly

Problem 4 — Race Condition: Redirect Before Token Restored

After fixing cookie, refresh returned 200. But page still went to login briefly.

Root Cause:

- ProtectedRoute checks accessToken immediately on load

- At that moment: accessToken = null (useEffect not finished yet)
- ProtectedRoute redirects to /login before refresh completes

Fix Applied:

- Added isLoading state to AuthContext — starts as true
- ProtectedRoute shows 'Loading...' while isLoading is true
- After refresh completes (success or fail) → setIsLoading(false)
- ProtectedRoute now waits for refresh to finish before deciding
- finally block ensures isLoading always becomes false

Problem 5 — Name/Email/Role Missing After Refresh

Dashboard showed empty name and email after browser reload. Token restored but user data was missing.

Root Cause:

- /refresh endpoint was only returning accessToken
- User object was not included in response
- After reload, setUser was never called

Fix Applied — 3 files changed:

- authService.js → Added user object to refresh return
- authController.js → Added user to refresh API response
- AuthContext.jsx → Destructured user from res.data.data and called setUser(user)

Problem 6 — Rate Limit Blocking Refresh After 9 Reloads

After 9-10 browser refreshes, user was logged out. Refresh endpoint returned 429 Too Many Requests.

Root Cause:

- All /auth/* routes shared same rate limiter
- Limit was 10 requests per 15 minutes
- Every Cmd+R used one request from this limit
- After 10 refreshes → blocked

Fix Applied — server.js:

- Created separate authLimiter: 10 requests / 15 min (for login + signup only)
- Created separate refreshLimiter: 60 requests / 15 min (for refresh route)
- Applied each limiter to specific route individually
- Login/signup still strictly limited — prevents brute force
- Refresh is generous — normal users can reload many times

Problem 7 — StrictMode Calling useEffect Twice

useEffect was calling /refresh twice on every load, wasting rate limit.

Fix Applied:

- Removed React.StrictMode from main.jsx
- useEffect now runs exactly once on startup

3. New Concepts Learned Today

Race Condition

When two things happen at same time and one finishes before the other is ready. Our fix: isLoading state makes ProtectedRoute WAIT for refresh to finish before deciding.

isLoading Pattern

A boolean flag that is true while async work is running. Components wait until it becomes false. Very common in production React applications.

sameSite Cookie Policy

Value	Meaning	When to Use
Strict	Cookie only sent on exact same site	High security, same domain only
Lax	Cookie sent on same site + some cross-site	Development, localhost
None	Cookie sent everywhere (needs secure:true)	Third-party integrations

HTTP vs HTTPS

Feature	HTTP	HTTPS
Encryption	No	Yes (SSL/TLS)
When to use	localhost development	Production / internet
Our code	NODE_ENV = development	NODE_ENV = production
secure cookie	false	true (auto)

Rate Limiting Strategy

Route	Limit	Reason
/auth/signup	10 / 15 min	Prevent spam account creation
/auth/login	10 / 15 min	Prevent brute force password attacks

/auth/refresh	60 / 15 min	Normal users reload browser many times
---------------	-------------	--

mongosh Commands Learned

mongosh	→ open MongoDB shell
use smart-hiring-portal	→ switch to database
db.users.find().pretty()	→ show all users nicely
db.users.countDocuments()	→ count total users

MongoDB Data Verified

9 users found in database. All passwords stored as bcrypt hash. refreshToken field stores hashed token or null. Admin user exists with role: admin (created directly in DB — cannot be created via API).

4. Files Changed Today

File	What Changed
backend/server.js	Split single authLimiter into authLimiter + refreshLimiter. Applied separately to routes.
backend/services/authService.js	Fixed null check order in refresh function. Added user object to refresh return. Removed all debug console.log lines.
backend/controllers/authController.js	Changed sameSite: Strict to sameSite: Lax. Added user to refresh response. Added req.cookies fallback.
frontend/src/context/AuthContext.jsx	Added isLoading state. Added useEffect for auto-refresh on startup. Added setUser after refresh. Added finally block.
frontend/src/services/authService.js	Changed baseUrl from 127.0.0.1:8000 to localhost:8000. Exported API instance for reuse.
frontend/src/App.jsx	Added isLoading check in ProtectedRoute. Shows Loading... while refresh is running. Full rewrite with clean structure.
frontend/src/main.jsx	Removed React.StrictMode to prevent double useEffect calls.

5. Complete Test Results

Test	Action	Result
Test 1	Login with email + password	Dashboard shows name, email, role correctly
Test 2	Press Cmd+R on dashboard	Dashboard stays — NOT redirected to login
Test 3	Go to /dashboard without login	Redirected to /login automatically
Test 4	Logout then go to /dashboard	Redirected to /login — session cleared
Test 5	Press Cmd+R 15+ times	Dashboard stays every time — no 429 error
Test 6	Login with wrong password	Red error: Invalid credentials
Test 7	Signup with existing email	Red error: Email already registered

Test 8	Check MongoDB users	9 users visible — passwords hashed — refreshToken stored
--------	---------------------	--

6. GitHub Commits Today

Commit Message	Files Changed
Fix: Separate rate limiters, complete auth lifecycle	7 files — server.js, authService.js, authController.js, AuthContext.jsx, authService.js, App.jsx, main.jsx
Fix: Complete auth lifecycle - refresh token working, rate limit fixed	1 file — authService.js debug logs removed
Fix: Complete auth lifecycle - refresh token working, rate limit fixed	1 file — final cleanup

7. EOD Summary

Task	Status
Fix browser refresh logout issue	Done
Debug and fix /auth/refresh 500 error	Done
Fix cookie cross-origin issue	Done
Add isLoading guard to ProtectedRoute	Done
Restore user data after browser refresh	Done
Fix rate limit blocking normal refresh	Done
Remove StrictMode double-call issue	Done
Verify MongoDB data using mongosh and Compass	Done
Remove debug console.log lines	Done
GitHub push with clean commits	Done

8. Key Lessons From Today

- Working code does not mean complete code — always test edge cases like page refresh
- Order of checks matters — always verify user exists before reading user properties
- Cookie hostname must match exactly — localhost and 127.0.0.1 are different for cookies
- Race conditions are common in async React — isLoading pattern solves this
- Rate limiters must be granular — different routes need different limits
- Debug logs are tools for finding bugs — remove them from final code
- mongosh and MongoDB Compass are essential tools for verifying database state
- HTTP is safe for localhost. HTTPS is mandatory for production internet deployment

9. Tomorrow — Module 2 Plan

- Module 2: User Profile Management
- Candidate profile — name, skills, bio, location
- Company profile — company name, industry, website
- Resume upload using Multer
- Profile edit functionality
- Profile view page

End of Day 5 Documentation | Smart Hiring Portal Internship | Mageswaran G